



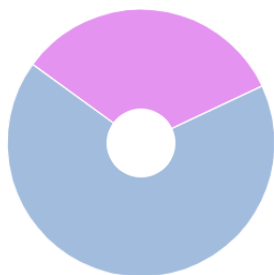
Отчет о проверке

Автор: bvbycccgff123123@mail.ru / ID: 13054714

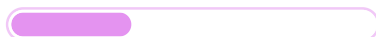
Проверяющий: (bvbycccgff123123@mail.ru / ID: 13054714)

Имя документа: Дипломдық жұмыс Сахиев Ж.У.

РЕЗУЛЬТАТЫ ПРОВЕРКИ



Машинно сгенерированный текст:
32,7%



Текст, написанный человеком:
67,3%



 Ряд характеристик этого текста указывают на то, что текст, возможно, был сгенерирован при помощи искусственного интеллекта. Однако, окончательное решение о качестве текста остается за проверяющим экспертом

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

Номер документа: 5

Тип документа: Не указано

Дата проверки: 04.05.2026 11:33:02

Дата корректировки: Нет

Количество страниц: 47

Символов в тексте: 78120

Слов в тексте: 8784

Число предложений: 2350

ИСХОДНЫЙ ТЕКСТ

Қазақстан Республикасының ғылым және жоғарғы білім министрлігі

А. Байтұрсынұлы атындағы Қостанай өңірлік университеті

«Қорғанысқа жіберілді»

Ақпараттық жүйелер

кафедрасының меңгерушісі

_____ Б.Кузенбаев

___ _____ 20__ г.

ДИПЛОМДЫҚ ЖҰМЫС

Тақырып: «Qazaq IT Academy» платформасында жасанды интеллектті

интеграциялау негізінде онлайн-білім беру жүйесін әзірлеу

білім беру бағдарламасына сәйкес

6B06102-Ақпараттық жүйелер

Қостанай, 2026

Орындады: Сахиев Ж.У, күндізгі оқу нысанының

4 курс студентті

Ғылыми жетекші: Карагожина А.К., аға оқытушы

2

Мазмұны

Кіріспе.....

1 Жасанды интеллект негізіндегі онлайн-білім беру жүйелерінің теориялық негіздері.....

1.1 Заманауи онлайн білім беру платформаларын талдау.....

1.2 Қолданбаның функционалдығына қойылатын талаптар.....

1.3 Бағдарламалық өнімді енгізу технологияларын таңдау.....

1.4 Жүйенің архитектурасы және деректер моделі.....

2 «Qazaq IT Academy» платформасының жобалық-техникалық шешімдері.....

2.1 Жүйенің жалпы архитектурасы және компоненттердің өзара байланысы.....

2.2 Клиенттік бөлік(Фронтенд): архитектуралық шешімдер және интерфейс концепциясы.....

2.3 Серверлік бөлік(Бэкенд): техникалық іске асыру, деректер базасы және қауіпсіздік.....

2.4 Жасанды интеллект модульдерінің бағдарламалық-техникалық реализациясы.....

3 Пайдаланушы нұсқаулығы.....

3.1 Студентке арналған нұсқаулық: платформамен толық жұмыс циклі және ИИ мүмкіндіктерін пайдалану.....

3.2 Басқа пайдаланушыларға арналған нұсқаулық: мұғалім, куратор, әкімші, директор және ата-ана.....

Қорытынды.....

Пайдаланылған әдебиеттер тізімі.....

3

Кіріспе

Қазіргі таңда цифрлық технологиялардың қарқынды дамуы білім беру

саласына түбегейлі өзгерістер енгізуде. Әсіресе онлайн-білім беру жүйелері

қоғамның барлық қабаттары үшін қолжетімді білім алудың тиімді құралына

айналды. Бұл үрдіс қашықтан оқыту форматының кеңінен таралуымен, сондай-ақ білім беру процесін автоматтандыру қажеттілігімен тікелей байланысты. Осы тұрғыда жасанды интеллект технологияларын білім беру платформаларына енгізу – өзекті әрі болашағы зор бағыттардың бірі болып табылады.

Жасанды интеллект негізіндегі жүйелер оқыту процесін дербестендіруге, студенттердің білім деңгейін талдауға және оларға жеке оқу траекториясын ұсынуға мүмкіндік береді. Сонымен қатар, мұндай жүйелер оқытушылардың жүктемесін азайтып, оқу материалдарын автоматты түрде тексеру, бағалау және кері байланыс беру сияқты функцияларды жүзеге асырады. Сондықтан қазіргі заманғы білім беру платформаларын әзірлеуде интеллектуалды технологияларды қолдану маңызды талаптардың біріне айналып отыр.

Зерттеу тақырыбының өзектілігі - оқу материалын жекелендіру, оқу процесін автоматтандыру және оқыту сапасын арттыру мақсатында онлайн-білім беру жүйелеріне жасанды интеллект технологияларын енгізу қажеттілігімен анықталады.

Дипломдық жұмыстың ғылыми жаңалығы - студенттердің оқу белсенділігін талдау, оқытуды жекелендіру, интеллектуалды қолдау және автоматтандырылған бағалауды қамтамасыз ететін онлайн-білім беру платформасының негізгі модулі ретінде жасанды интеллектті енгізумен анықталады.

Дипломдық жұмыстың мақсаты - «Qazaq IT Academy» платформасы негізінде жасанды интеллект технологияларын интеграциялау арқылы заманауи онлайн-білім беру жүйесін жобалау және әзірлеу болып табылады.

Жұмыстың негізгі міндеттері ретінде келесілер қарастырылды:

- онлайн-білім беру жүйелерінің теориялық негіздерін зерттеу;
- платформаның архитектурасын жобалау және технологиялық стек таңдау;
- пайдаланушыларға арналған интерфейстер мен функционалдық модульдерді әзірлеу;

- жасанды интеллектті жүйеге интеграциялау және оның мүмкіндіктерін іске асыру;

- жүйенің қауіпсіздігін, өнімділігін және тұрақтылығын қамтамасыз ету.

Зерттеу нысаны – онлайн оқытуды ұйымдастыру жүйесі, ал зерттеу пәні жасанды интеллект технологияларына негізделген білім беру веб-платформасының архитектурасы мен функционалдығы.

Зерттеу әдістері - дипломдық жұмысты орындау барысында келесі зерттеу әдістері қолданылды:

- ғылыми әдебиеттер мен ақпарат көздерін талдау;
- салыстырмалы және жүйелік талдау әдістері;
- объектіге бағытталған жобалау (UML-диаграммалар);
- бағдарламалық модельдеу және жүйені құрастыру;

4

- эмпирикалық тестілеу;
- алынған нәтижелерді талдау және өңдеу.

Бұл әдістер зерттеу нәтижелерінің дәлдігі мен сенімділігін қамтамасыз етуге мүмкіндік берді.

Дипломдық жұмыстың теориялық маңыздылығы - онлайн-білім беру жүйелерінде жасанды интеллект технологияларын қолдану мүмкіндіктерін, адаптивті оқыту модельдерін және оларды білім беру платформаларына интеграциялау тәсілдерін жүйелеумен анықталады.

Зерттеу нәтижелері - болашақта онлайн білім беру платформаларын

жобалау және жетілдіру кезінде, сондай-ақ білім беру саласында жасанды интеллектті қолдану саласындағы ғылыми зерттеулерде пайдаланылуы мүмкін. Жобаның практикалық маңыздылығы – ұсынылған жүйе білім беру процесін автоматтандыруға, студенттердің оқу белсенділігін арттыруға және оқыту сапасын жақсартуға бағытталған нақты жұмыс істейтін платформа болып табылады. Сонымен қатар, бұл жүйе оқу орталықтарында немесе IT академияларда қолдануға бейімделген. Дипломдық жұмыс мазмұндық тұрғыдан жүйеленіп, кіріспе бөлімнен, негізгі үш тараудан, қорытынды бөлімнен және пайдаланылған дереккөздер тізімінен құралған. Бірінші бөлімде теориялық негіздер қарастырылса, екінші бөлімде жүйені жобалау және техникалық жүзеге асыру сипатталады, ал үшінші бөлімде пайдаланушы нұсқаулығы ұсынылған.

5

1 Жасанды интеллект негізіндегі онлайн-білім беру жүйелерінің теориялық негіздері

1.1 Заманауи онлайн білім беру платформаларын талдау

Соңғы жылдары білім беру саласында онлайн-платформалардың рөлі айтарлықтай артты. Интернет технологияларының дамуы мен қашықтан оқытудың кең таралуы білім беру жүйесін цифрландыру үдерісін жеделдетті. Қазіргі уақытта көптеген білім беру ұйымдары оқу материалдарын ұсыну, тестілеу жүргізу және білімді бағалау мақсатында онлайн-платформаларды кеңінен қолданады.

Қолданыстағы онлайн-білім беру жүйелерін салыстырмалы талдау мақсатында кең таралған үш платформа зерттелді: Coursera, edX және Khan Academy. Аталған платформалар оқу процесін ұйымдастыруды жеңілдеткенімен, олардың бірқатар шектеулері анықталды.

Coursera — АҚШ-тың Stanford университеті негізінде 2012 жылы құрылған, әлемдік деңгейдегі ең ірі MOOC-платформаларының бірі. Платформада 7000-нан астам курс ұсынылады, жетекші университеттермен серіктестік орнатылған. Алайда оқыту мазмұны негізінен сызықтық түрде ұсынылады, пайдаланушылардың жеке оқу қарқыны мен деңгейі жеткілікті деңгейде ескерілмейді. Сонымен қатар интерфейс пен контент басым бөлігінде ағылшын тілінде ұсынылады, бұл қазақстандық аудитория үшін қолжетімділікті төмендетеді.

edX — MIT және Harvard университеттері бірлесіп 2012 жылы іске қосқан платформа. Ашық бастапқы кодқа (Open edX) негізделген, бұл институттарға өз платформасын қосымша баптауға мүмкіндік береді. Дегенмен дараландырылған оқыту механизмдерінің жеткіліксіздігі әр студент үшін жеке оқу траекториясын қалыптастыруды қиындатады. Бағалау және кері байланыс процестері толық автоматтандырылмаған.

Khan Academy — бастауыш және орта білімге бағытталған, тегін білім беру платформасы. Платформада адаптивті жаттығулар элементтері бар, алайда жоғары оқу орындары мен кәсіби IT-оқыту форматына арналмаған.

Мұғалімдермен тікелей байланыс мүмкіндігі шектеулі.

Жүргізілген салыстырмалы талдау нәтижелері төмендегі 1-кестеде жинақталған:

1-кесте — Онлайн-білім беру платформаларының салыстырмалы талдауы

Критерий Coursera edX Khan Academy Qazaq IT Academy

Жекелендірілген

оқыту

Жартылай Жартылай Бар Бар (LLM API

негізінде)

6

Жүргізілген талдау нәтижесінде қолданыстағы платформалардың

бейімделгіштік және интеллектуалды мүмкіндіктері жеткіліксіз екені, сонымен

қатар олардың Қазақстан білім беру жүйесінің тілдік және әдістемелік

ерекшеліктерін толық ескермейтіні анықталды. Осыған байланысты оқу

процесін дараландыруға, барлық қатысушы рөлдерін толық қамтуға және

жасанды интеллектті негізгі функционалдық модуль ретінде пайдалануға

мүмкіндік беретін жергілікті білім беру платформасын әзірлеу қажеттілігі

туындайды [13, 14].

Аталған мәселелерді шешудің тиімді жолы — онлайн-білім беру жүйелеріне

жасанды интеллект технологияларын енгізу болып табылады. Жасанды

интеллект пайдаланушылардың оқу әрекеттерін талдауға, оқу материалдарын

бейімдеуге және автоматты ұсыныстар қалыптастыруға мүмкіндік береді [2, 3].

1.2 Қолданбаның функционалдығына қойылатын талаптар

Алдыңғы бөлімде жүргізілген талдау нәтижесінде қолданыстағы онлайн-

білім беру жүйелерінің бірқатар шектеулері анықталды. Осыған байланысты

жаңа интеллектуалды білім беру платформасын әзірлеу барысында жүйеге

ЖИ-ассистент Кездеспейд

i

Кездеспейді Кездеспейді Бар (іске

асырылған)

Қазақ тілі қолдауы Кездеспейд

i

Кездеспейді Кездеспейді Бар

Рөлдік басқару жүйесі Шектеулі Шектеулі Шектеулі Бар (6 рөл, RBAC)

Ата-аналық бақылау Кездеспейд

i

Кездеспейді Кездеспейді Бар (іске

асырылған)

Геймификация Кездеспейд

i

Кездеспейді Кездеспейді Бар (Q-coin жүйесі)

Автоматты тексеру Жартылай Жартылай Кездеспейді Жартылай (LLM

арқылы, мұғалім

растайды)

Жергілікті төлем

жүйесі

Кездеспейд

i

Кездеспейді Кездеспейді Бар (Kaspi Pay)

Масштабталу

мүмкіндігі

Жеткілікті

деңгейде

Жеткілікті

деңгейде

Жеткілікті

деңгейде

қойылатын функционалдық талаптарды нақты анықтау маңызды болып табылады.

Болашақ платформа пайдаланушыларға ыңғайлы интерфейс ұсынумен қатар, оқу процесін автоматтандыруды және дараландыруды қамтамасыз етуі тиіс.

Жүйенің негізгі функционалдық талаптары пайдаланушылардың рөлдеріне байланысты анықталады. Платформада алты негізгі пайдаланушы рөлі қарастырылады: студент, оқытушы, куратор, әкімші, директор және ата-ана. Рөлдерді толық алдын ала анықтау жүйенің қолжетімділік матрицасын дұрыс жобалауға және жоба барысында архитектуралық шешімдерді дәйекті қабылдауға мүмкіндік береді.

Студент үшін функционалдық мүмкіндіктер:

- жүйеге кіру мүмкіндігі (аккаунт Kaspi Pay арқылы сатып алу немесе әкімші тіркеуі арқылы жасалады);
- оқу курстарын таңдау және оқу материалдарын қарау;
- тапсырмаларды орындау және тестілеуден өту;

• оқу нәтижелерін көру және жеке прогресті бақылау;

• жасанды интеллект ассистентімен жұмыс істеу және жеке оқу

траекториясын пайдалану;

• Q-coin жинау және Q Market дүкенінде жұмсау;

• AI Challenge жарысына қатысу.

Оқытушы үшін функционалдық мүмкіндіктер:

• оқу курстарын құру және өңдеу;

• оқу материалдарын жүктеу және жаңарту;

• тапсырмалар мен тест материалдарын әзірлеу;

• студенттердің нәтижелерін бақылау және бағалау;

• оқу процесіне қатысты кері байланыс беру;

• жасанды интеллект ұсынған бастапқы бағалауды пайдалану.

Куратор үшін функционалдық мүмкіндіктер:

• оқытушымен бірдей педагогикалық функцияларды орындау;

• топтарды басқару: студенттерді топқа қосу және шығару;

• топқа қосылу өтінімдерін өңдеу және растау;

• студенттердің жалпы үлгерімін қадағалау.

Әкімші үшін функционалдық мүмкіндіктер:

• пайдаланушыларды басқару: тіркеу, рөл тағайындау, өзгерту;

• курстарды бақылау және жүйелік параметрлерді реттеу;

• жүйенің жұмысын мониторингтеу;

• деректердің қауіпсіздігін қамтамасыз ету;

• аналитика деректерін Excel форматында экспорттау.

Директор үшін функционалдық мүмкіндіктер:

• жүйелік басқару панеліне толық қолжетімділік;

• платформа статистикасын жалпы түрде бақылау;

• қолдау тикеттерін өңдеу.

Ата-ана үшін функционалдық мүмкіндіктер:

• баланың оқу прогресін қашықтан бақылау;

• орташа балл, меңгерілген тақырыптар саны, рейтингтегі орын туралы ақпаратты көру;

- мерзімі өткен және жақындаған тапсырмалар туралы хабарландыру алу;
- үлгерім есебін PDF форматында жүктеу.

Жүйенің маңызды ерекшеліктерінің бірі — жасанды интеллект модулінің болуы. Бұл модуль платформаның жеке функционалдық қабаты ретінде қарастырылып, барлық интеллектуалды функцияларды біріктіруге арналған.

Жасанды интеллект модулінің негізгі функциялары:

- пайдаланушылардың оқу белсенділігін талдау;
- студенттердің білім деңгейін бағалау және жеке ұсыныстар қалыптастыру;
- интеллектуалды ассистент арқылы сұрақтарға контекстік жауап беру;
- ашық жауаптарды автоматты түрде тексеру және мұғалімге бастапқы бағалау ұсыну;
- оқу нәтижелері бойынша аналитикалық есептер қалыптастыру;
- жеке оқу жоспарын генерациялау;
- AI Challenge жарысында сұрақтарды динамикалық генерациялау.

Маңызды ескерту: жобаның аясында дербес машиналық оқыту модельдері оқытылған жоқ. Жасанды интеллект функциялары сыртқы LLM API-сервистері (OpenAI GPT-4o және Google Gemini 1.5 Flash) және студент прогресін бағалауға арналған эвристикалық алгоритмдер (прогресс деңгейі, оқу қарқыны, тест нәтижелері) арқылы іске асырылған. Болашақта бұл модульге нейрожелілік модельдерді енгізу арқылы жүйенің мүмкіндіктерін кеңейтуге болады. Сонымен қатар, жүйеде деректерді сақтау, өңдеу және қауіпсіздігін қамтамасыз ету функциялары қарастырылуы тиіс. Пайдаланушылардың деректері қорғалып, жүйеге рұқсатсыз қол жеткізу әрекеттерінен сақталуы қажет.

Жүйеге қойылатын талаптар екі негізгі топқа бөлінеді: функционалдық және функционалдық емес.

Функционалдық талаптар:

Жүйеде тіркелу тек әкімші немесе оқытушы арқылы жүзеге асады, ал ашық тіркелу шектелген. Авторизация FastAPI деңгейінде JWT арқылы орындалып, құпия сөздер bcrypt көмегімен қорғалады. Рөлдер (студент, оқытушы, куратор, әкімші, директор, ата-ана) RBAC арқылы бөлінеді. Курстар жүйесі бейне, мәтін және тест материалдарын қамтиды, ал студенттер оларды санат бойынша іздей алады. Прогресс автоматты түрде есептеледі. ЖИ модулі оқу тарихын талдап, жеке жоспар құрады және виртуалды ассистент ретінде жауап береді. Сонымен қатар эссе мен тесттерді автоматты тексеру мүмкіндігі бар.

Тестілеу жүйесі әртүрлі форматтағы тапсырмаларды қолдайды және нәтижелер автоматты түрде бағаланады. Коммуникация бөлімі хабарландыру жүйесі мен қолдау тикет жүйесін қамтиды.

Функционалдық емес талаптар:

Жобалық талап ретінде жүйе бір уақытта шамамен 1000 пайдаланушыны қолдауы, ал API жауап уақыты 500 мс-тан аспауы белгіленген. Қауіпсіздік HTTPS арқылы қамтамасыз етіледі, деректер қорында құпия сөздер хэштеледі, сондай-ақ

SQL инъекция мен XSS шабуылдарынан қорғаныс қарастырылған. Жобалық қолжетімділік деңгейі 99.9% болуы тиіс, ал резервтік көшіру күн сайын автоматты түрде орындалады. Бұл мәндер жобалық талаптар ретінде белгіленген; нақты тестілеу нәтижелері жүйенің өндірістік ортада жұмыс істеу барысында тексерілуі тиіс.

Бағдарламалық өнімді әзірлеудің негізгі тұжырымдамалары мен

кезеңдері

Бағдарламалық өнім үш негізгі қағидаға сүйеніп жасалған. Біріншісі —

пайдаланушыға бағытталған дизайн, яғни жүйе студент пен оқытушының нақты

қажеттіліктеріне бейімделген. Екіншісі — адаптивті оқыту, мұнда жасанды

интеллект әр студентке жеке оқу траекториясын ұсынады. Үшіншісі — модульдік

құрылым, ол жүйені бөлек бөліктер ретінде дамытуға және оңай жаңартуға

мүмкіндік береді.

Бағдарламалық өнімді әзірлеу процесі келесі кезеңдерден тұрады:

Бірінші кезең — Талдау және жоспарлау. Бұл кезеңде қолданушылардың

талаптары зерттеліп, нарықтағы бәсекелес платформалар талданады.

Функционалдық және функционалдық емес талаптар анықталады, жобаның

техникалық-экономикалық негіздемесі жасалады.

Екінші кезең — Жүйені жобалау. Бұл кезеңде деректер базасының

схемасы, серверлік архитектура және пайдаланушы интерфейсінің прототиптері

әзірленеді. Жасанды интеллект модульдерін интеграциялау логикасы

анықталады.

Үшінші кезең — Әзірлеу және бағдарламалау. Frontend және backend

бөліктері параллель әзірленеді. Жасанды интеллект алгоритмдері платформаға

интеграцияланады, деректер базасы конфигурацияланады.

Төртінші кезең — Тестілеу және сапаны қамтамасыз ету.

Функционалдық тестілеу, жүктеме тестілеу және қауіпсіздік тестілеу жүргізіледі.

Пайдаланушылармен бета-тестілеу өткізіледі.

Бесінші кезең — Енгізу және техникалық қолдау. Платформа нақты

пайдаланушыларға ұсынылады, жүйенің тұрақты жұмыс істеуі қамтамасыз

етіледі, кері байланыс негізінде жетілдірулер енгізіледі.

1.3 Бағдарламалық өнімді енгізу технологияларын таңдау

Интеллектуалды білім беру платформасын әзірлеу барысында

қолданылатын технологияларды дұрыс таңдау жүйенің өнімділігіне, кеңейтілу

мүмкіндігіне және тұрақты жұмысына тікелей әсер етеді. Осыған байланысты

серверлік бөлік, клиенттік интерфейс, деректер базасы және жасанды интеллект

модулін жүзеге асыру үшін бірнеше технологиялар салыстырылып талданды.

Backend технологиялары. Жүйенің серверлік бөлігін жүзеге асыру үшін

FastAPI, Django және Node.js технологиялары қарастырылды.

Django фреймворкі толық функционалды веб-жүйелерді құруға ыңғайлы

және әкімшілік панелі дайын түрде ұсынылады. Алайда оның құрылымы

салыстырмалы түрде күрделі және кейбір жағдайларда артық ресурстарды талап

етуі мүмкін. Node.js серверлік қосымшаларды жылдам әзірлеуге мүмкіндік

10

береді және JavaScript тілін қолдануға негізделген. Дегенмен, күрделі деректерді

өңдеу және жасанды интеллект алгоритмдерімен интеграция кезінде Python

тілінің экожүйесі анағұрлым тиімді болып табылады.

Осыған байланысты серверлік бөлік үшін FastAPI фреймворкі таңдалды.

FastAPI асинхронды өңдеуді қолдайды, жоғары өнімділікке ие және Python

тілінде жасалған. Бұл жасанды интеллект API-сервистерімен серверлік логиканы

біріктіруді жеңілдетеді. Сонымен қатар FastAPI REST API қызметтерін құруға

ыңғайлы, автоматты OpenAPI құжаттамасын генерациялайды және болашақта

жүйені кеңейтуге мүмкіндік береді. Серверді іске қосу үшін Uvicorn

қолданылады, ол ASGI стандартын пайдаланып сұраныстарды жылдам өңдеуді

қамтамасыз етеді

1-сурет – Backend технологиялары: FastAPI және Uvicorn архитектурасы

Деректер базасы. Деректер базасын таңдау барысында PostgreSQL, MySQL

және MongoDB жүйелері қарастырылды.

MySQL кеңінен қолданылатын және қарапайым жобалар үшін ыңғайлы

жүйе болып табылады. MongoDB құжаттық деректерді сақтау үшін тиімді

болғанымен, құрылымы тұрақты реляциялық деректер үшін кейбір шектеулерге

ие болуы мүмкін. Ал PostgreSQL күрделі құрылымдағы деректермен жұмыс

істеуге арналған сенімді жүйе болып табылады және транзакцияларды қолдау

мүмкіндігі жоғары деңгейде жүзеге асырылған. PostgreSQL деректер базасы

үлкен көлемдегі ақпаратты тиімді өңдеп, пайдаланушылар, курстар және

нәтижелер деректерін басқаруға қолайлы болғандықтан таңдалды.

Деректер базасымен жұмысты оңтайландыру үшін SQLAlchemy ORM

қолданылады — кестелер Python кластары ретінде модельденген, бұл SQL-

инъекция шабуылдарынан қорғауды да қамтамасыз етеді.

2-сурет – Деректер базасы: PostgreSQL және SQLAlchemy ORM интеграциясы

Frontend технологиялары. Клиенттік интерфейсті жүзеге асыру үшін

React, Angular және Vue.js технологиялары қарастырылды.

Angular күрделі корпоративтік жүйелер үшін қолайлы болғанымен, оны

меңгеру салыстырмалы түрде күрделі болып табылады. Vue.js жеңіл және

қарапайым интерфейстер үшін тиімді, алайда үлкен жобаларда кеңейту

мүмкіндіктері шектеулі болуы мүмкін. React кітапханасы компоненттік

құрылымды қолдануға мүмкіндік береді және интерфейсті жылдам жаңартуға

11

қолайлы. Пайдаланушы интерфейсін құру үшін Next.js 16 (React 19 негізінде)

және TypeScript таңдалды. Next.js серверлік рендеринг (SSR) және статикалық

генерация (SSG) мүмкіндіктері арқасында жылдамдық пен SEO тиімділігі

қамтамасыз етіледі. Стильдеуге Tailwind CSS 4 пайдаланылды, ол жылдам

прототиптеуге және адаптивті дизайн жасауға мүмкіндік береді.

3-сурет – Frontend технологиялары: Next.js, React, TypeScript және Tailwind CSS

Жасанды интеллект технологиялары. Платформада жасанды интеллект

функцияларын іске асыру үшін сыртқы LLM API-сервистері таңдалды. Дербес

машиналық оқыту модельдерін (TensorFlow, PyTorch, scikit-learn) оқыту жоба

аясында жүзеге асырылған жоқ, өйткені заманауи LLM-сервистері жоғары

сапалы нәтижелерді жылдам және арнайы инфрақұрылымсыз ұсынуға мүмкіндік

береді.

Негізгі ретінде OpenAI API (GPT-4o) таңдалды — күрделі аналитика, код

талдау және ашық жауаптарды бағалау үшін. Резервтік ретінде Google Gemini

API (Gemini 1.5 Flash) орнатылған. Бұл тәсіл бір API қолжетімсіз болған

жағдайда жүйенің тұрақты жұмысын қамтамасыз етеді. Студент үлгерімін

болжау эвристикалық алгоритмдер (прогресс деңгейі, оқу қарқыны, тест

нәтижелерінің динамикасы) негізінде жүзеге асады.

Жүйелік орта. Платформа Docker контейнерлерінде орналастырылған.

docker-compose.vps.yml конфигурациясы бойынша backend, frontend және

PostgreSQL бөлек контейнерлерде жұмыс істейді. Нақты сервер ретінде VPS

(Virtual Private Server) қолданылады, бұл икемді баптауға және шығын

тиімділігіне мүмкіндік береді.

Төмендегі 2-кестеде таңдалған технологиялардың жиынтық сипаттамасы

берілген:

2-кесте — Таңдалған технологиялардың сипаттамасы

Технология Негізгі міндеті Басым тұстары

Next.js 16 + TypeScript Клиенттік бөлікті

дамыту

SSR/SSG қолдауы, типтік

қауіпсіздік

Python + FastAPI + Uvicorn Серверлік логиканы

құру

Асинхронды өңдеу, автоматты

құжаттама

PostgreSQL + SQLAlchemy Деректер базасы Реляциялық тұтастық, күрделі

сұраныстар

12

1.4 Жүйенің архитектурасы және деректер моделі

«Qazaq IT Academy» платформасының архитектурасы модульдік монолит

(modular monolith) түрінде жобаланған. Барлық бизнес-логика бір FastAPI

қосымшасында орналасқан, бірақ модульдер (аутентификация, курстар, тесттер,

ЖИ) логикалық тұрғыдан бөлінген. Бұл тәсіл әзірлеуді жеңілдетеді, тестілеуді

оңтайландырады және болашақта масштабталуды қамтамасыз етеді.

Архитектура клиент-сервер моделіне сәйкес үш деңгейден тұрады:

1. Презентациялық деңгей — Next.js клиенті (пайдаланушы

интерфейсі, SSR/SSG рендеринг);

2. Бизнес-логика деңгейі — FastAPI монолитті қосымша

(аутентификация, курстар, тесттер, ЖИ модульдері);

3. Деректер деңгейі — PostgreSQL реляциялық деректер базасы.

Серверлік бөлік FastAPI арқылы іске асып, аутентификацияны, деректерді

өңдеуді және REST API арқылы клиентпен байланысты қамтамасыз етеді.

Деректер PostgreSQL-де сақталып, SQLAlchemy ORM көмегімен өңделеді.

Деректер моделі PostgreSQL реляциялық СУБД негізінде құрылған.

Кестелер SQLAlchemy ORM көмегімен Python кластары ретінде модельденген (app/models/ каталогы). Жүйеде келесі негізгі кестелер анықталған: users, courses, groups, enrollments, topics, assignments, submissions, test_results, ai_chat_history, q_coin_transactions, enrollment_requests. Олардың арасындағы байланыс foreign key шектеулері арқылы қамтамасыз етіледі.

Платформаның жасанды интеллект жүйесі негізгі сұраныстарды өңдеп

жауап қалыптастырады, ал резервтік сервис қолжетімсіздік жағдайында

автоматты түрде іске қосылады. Арнайы алгоритм студенттің оқу қарқыны мен

жетістігін талдап, соған сай тапсырмалар мен курстарды ұсынады. Сонымен

қатар, жүйе ашық жауаптарды автоматты түрде тексеріп, мазмұнды кері

байланыс береді.

Осылайша, жүргізілген салыстырмалы талдау нәтижесінде серверлік бөлік

үшін FastAPI, деректер базасы үшін PostgreSQL, пайдаланушы интерфейсі үшін

Next.js (React негізінде) және жасанды интеллект функцияларын іске асыру үшін

OpenAI API мен Google Gemini API-сервистерін қолдану тиімді шешім болып

табылады. Таңдалған технологиялар жүйенің өнімділігін арттыруға, кеңейтілу

мүмкіндігін қамтамасыз етуге және интеллектуалды функцияларды тиімді

жүзеге асыруға мүмкіндік береді.

Анықталған талаптар мен таңдалған технологиялық стек негізінде екінші

тарауда платформаның архитектуралық және бағдарламалық-техникалық шешімдері егжей-тегжейлі сипатталады.

OpenAI API + Google Gemini

API

Жасанды интеллект Резервтік жүйе, жоғары

сенімділік

Docker + VPS Жобаның техникалық

құрылымы

Контейнерлеу, икемді

орналастыру

13

2 «Qazaq IT Academy» платформасының жобалық-техникалық

шешімдері

2.1 Жүйенің жалпы архитектурасы және компоненттердің өзара байланысы

«Qazaq IT Academy» платформасы заманауи клиент-серверлік архитектура

негізінде құрылған. Жүйе бірнеше дербес компоненттен тұрады: пайдаланушы

интерфейсі (frontend — клиенттік бөлік), серверлік бизнес-логика (backend —

серверлік бөлік), реляциялық деректер қоры және жасанды интеллект

қызметтері. Олардың әрқайсысы бөлек Docker контейнерінде орналасқан, бұл

жүйенің икемді орналастырылуын және оңай масштабталуын қамтамасыз етеді.

Архитектуралық тұрғыдан алғанда жүйе үш негізгі деңгейге бөлінеді.

Бірінші деңгей — презентациялық, мұнда Next.js негізіндегі клиенттік бөлік

пайдаланушымен тікелей байланысады және барлық визуалды интерфейсті

басқарады. Екінші деңгей — бизнес-логика деңгейі, мұнда FastAPI

фреймворкінде жазылған серверлік бөлік барлық сұраныстарды өңдейді,

деректерді тексереді және жасанды интеллект модульдерімен байланысады.

Үшінші деңгей — деректер деңгейі, мұнда PostgreSQL реляциялық деректер

қоры барлық тұрақты ақпаратты сақтайды.

5--сурет – Жүйенің жалпы архитектуралық схемасы: Frontend → Nginx → Backend →

PostgreSQL / AI APIs

Физикалық орналасу тұрғысынан барлық компоненттер бір VPS серверінде

Docker контейнерлері арқылы іске қосылады. Nginx кері прокси сервері сыртқы

трафикті қабылдап, SSL/HTTPS шифрлауды қамтамасыз ете отырып,

сұраныстарды тиісті контейнерге бағыттайды. Клиенттік бөлік 3000 портында,

серверлік бөлік 8000 портында жұмыс істейді, ал PostgreSQL тек ішкі Docker

желісінде қолжетімді болып, сыртқа ашылмайды. Бұл шешім деректер базасына

тікелей сыртқы қолжетімділікті шектеп, жүйенің қауіпсіздік деңгейін арттыруға

мүмкіндік береді.

6-сурет – Docker контейнерлерінің физикалық орналасу схемасы / docker ps

14

Компоненттердің өзара байланысы мынадай механизмдер арқылы жүзеге

асады. Клиенттік және серверлік бөліктер арасындағы байланыс REST API

протоколы арқылы орындалады. Next.js қосымшасында орталықтандырылған

Axios клиенті (src/api/client.ts) конфигурацияланған. Бұл клиент екі маңызды

интерцептор механизмін қамтиды: бірінші интерцептор әрбір сұранысқа JWT

токенін автоматты түрде Authorization: Bearer форматында қосады, ал екіншісі

сервер 401 қатесін қайтарған жағдайда пайдаланушыны кіру бетіне бірден

бағыттайды. CORS мәселелерін болдырмау үшін Next.js конфигурациясында

domain.com/api/* маршруттары тікелей серверлік контейнерге проксиленеді.

Деректер алмасу форматы ретінде JSON қолданылады. Нақты уақыт

режимін қажет ететін модульдерде — мысалы, ЖИ чатта — сервер жауабы

streaming арқылы беріледі, яғни жауап бөліктеп жіберіліп, пайдаланушы оны

бірден оқи алады.

Технологиялық стек жайында айтатын болсақ, клиенттік бөлік Next.js 16

(App Router) және TypeScript негізінде жасалған. Стильдеу Tailwind CSS 4

арқылы орындалып, адаптивті дизайн қамтамасыз етілген. Жаһандық күй басқару үшін Zustand, серверден деректер алу үшін TanStack Query (React Query) кітапханалары қолданылған. Серверлік бөлік Python 3.11+ тілінде FastAPI фреймворкімен жазылған. Асинхронды сұраныстарды өңдеу Uvicorn ASGI сервері арқылы жүзеге асады. Деректер базасымен байланыс SQLAlchemy ORM арқылы орындалады. Аутентификацияда python-jose кітапханасы, ал құпия сөздерді хэштеуде passlib/bcrypt қолданылады.

7-сурет – Технологиялық стектің жиынтық кестесі

Жүйенің негізгі API маршруттары келесідей анықталған: аутентификация үшін /api/v1/auth/login (POST), курстар тізімі үшін /api/v1/courses (GET), ЖИ чат үшін /api/v1/ai-bot/chat (POST), ЖИ жарысты бастау үшін /api/v1/challenge/start (POST), студент аналитикасы үшін /api/v1/analytics/student-insights/weak-topics (GET), ал төлем үшін /api/v1/payments/kaspi/pay (POST) маршруты қолданылады.

Деректер базасының негізгі кестелері мен байланыстары жайында айтатын болсақ, жүйеде users, courses, groups, enrollments, topics, assignments, submissions, test_results, ai_chat_history, q_coin_transactions, enrollment_requests сияқты негізгі кестелер бар. Олардың арасындағы байланыс foreign key шектеулері арқылы қамтамасыз етіледі. Мысалы, enrollments кестесі user_id және group_id өрістері арқылы пайдаланушы мен топты байланыстырады. Деректер базасын жаңарту миграциялар арқылы орындалады, бұл схеманың тарихи өзгерістерін бақылауға мүмкіндік береді.

15

8-сурет – ER-диаграмма: негізгі кестелер мен байланыстар

Платформаны болашақта масштабтау мәселесіне келетін болсақ, қазіргі архитектура жобалық талап ретінде шамамен 500–1000 бір уақытта жұмыс істейтін пайдаланушыны қолдауға есептелген. Бұл шекке жеткенде бірнеше стратегия қолдануға болады.

Бірінші стратегия — горизонталды масштабтау. FastAPI серверлік бөлігінің бірнеше инстансын іске қосып, олардың алдына Nginx load balancer орнату арқылы жүктемені бөлуге болады. Docker Compose конфигурациясын Kubernetes кластеріне көшіру бұл процесті автоматтандырады.

Екінші стратегия — кэштеу деңгейін қосу. Redis кэш сервері орнатылса, жиі сұралатын деректер (курстар тізімі, рейтинг кестесі, пайдаланушы профилі) деректер базасына қайта-қайта сұраныс жібермей оперативті жадтан алынады.

Бұл деректер базасының жүктемесін айтарлықтай азайтуы мүмкін.

Үшінші стратегия — деректер базасын оңтайландыру. Қолданушылар саны ұлғайған жағдайда PostgreSQL replica (оқу репликасы) орнату ұсынылады.

Бұл жағдайда оқу сұраныстары репликаға, ал жазу сұраныстары негізгі серверге бағытталады. Сонымен қатар жиі қолданылатын өрістерге индекстер қою (user_id, group_id, created_at) сұраныстардың орындалу жылдамдығын арттырады.

Төртінші стратегия — медиа файлдарды бөлек сақтау. Қазіргі уақытта жүктелген файлдар сервердің volumes ішінде сақталады. Болашақта S3-үйлесімді объект қоймасына (мысалы, MinIO немесе AWS S3) көшу сервер дискісіндегі жүктемені азайтады және файлдарға CDN арқылы жылдам қолжетімділікті қамтамасыз етеді.

ЖИ сұраныстарының масштабтануына келетін болсақ, OpenAI мен Gemini API-лары сыртқы қызметтер болғандықтан, олардың жүктемесі платформаның өз серверіне тікелей түспейді. Алайда API шектеулеріне (rate limits) тап болмай үшін сұраныстарды кезекке (queue) қою механизмін енгізу ұсынылады.

9-сурет – Патформаның болашақта масштабталатын архитектуралық схемасы

2.2 Клиенттік бөлік(Фронтенд): архитектуралық шешімдер және

интерфейс концепциясы

Платформаның клиенттік бөлігі Next.js 16 фреймворкінің App Router

мүмкіндіктерін толық пайдалана отырып жасалған. Интерфейс барлық рөлдер үшін бір кодтық база шеңберінде ұйымдастырылған, бірақ пайдаланушының рөліне қарай әртүрлі компоненттер мен беттерді көрсетеді. Рөлге негізделген маршруттау (RBAC) /app директориясы ішінде жүзеге асырылған: /app/student, /app/teacher, /app/admin сияқты жолдар серверлік middleware арқылы қорғалған. Жалпы беттер саны 40-тан асады.

Басты бет және пайдаланушы интерфейсі

Басты бет платформаның алғашқы беті болып табылады. Hero-секцияда `TypingHeroText` компоненті арқылы мәтін анимациясы орындалады — бұл пайдаланушының назарын бірден аударып, платформаның бағытын бір сөйлеммен жеткізеді.

Курстар каталогі клиент жағында санаттар бойынша динамикалық сүзгіленеді. Бұл тәсілдің техникалық маңызы зор: бет толық қайта жүктелмей-ақ мазмұн бірден өзгереді, яғни серверге артық сұраныс жіберілмейді және пайдаланушы тәжірибесі жылдамдайды. Сүзгілеу логикасы `React state` арқылы басқарылып, `useState` хуктарымен іске асырылған.

Навигациялық панель бетті айналдырғанда жоғарыда бекіп қалады (`sticky positioning`) және мөлдірлік эффектісімен (`backdrop-filter: blur`) безендірілген.

Интерфейс үш тілді — қазақ, орыс, ағылшын — толық қолдайды. Тілді ауыстыру жоғарғы навигациядан бір батырма арқылы орындалады және таңдау `localStorage`-та сақталады. Ашық және қараңғы тақырыптар арасында ауысу мүмкіндігі де бар — тақырып параметрі де браузерде сақталады.

17

10-суреттер - Басты беттің Hero-секциясы және курстар каталогінің динамикалық сүзгілеу панелі

11-сурет - Тіл ауыстыру мәзірі

Авторизация беті қарапайым, функционалды интерфейспен жасалған.

Форма клиент жағында `Zod` кітапханасы арқылы тексеріледі: `email` форматы, бос өрістер сервер сұранысы жіберілмей-ақ бірден белгіленеді, бұл серверге түсетін артық жүктемені азайтады. Авторизациядан кейін `JWT` токени `cookies`-та сақталады, ал `Zustand persist middleware` арқылы аутентификация күйі браузер сессиялары арасында жоғалмайды.

12-сурет – Авторизация бетінің интерфейсі

Студент жұмыс кеңістігі

Студент дашборды авторизациядан кейін ашылатын негізгі жұмыс кеңістігі

болып табылады. Дашбордта соңғы оқылған курс пен тоқтаған сабақ автоматты түрде анықталып, «Жалғастыру» батырмасы арқылы пайдаланушы бір кликпен оқуға оралады. Бұл шешім пайдаланушының оқу процесіне кедергісіз оралуын қамтамасыз ету мақсатында іске асырылған.

Платформада геймификация жүйесі енгізілген: `Q-coin` валютасы, `Q-Market`

дүкені және үш деңгейлі `Leaderboard (Top / Middle / Low)`. `Q-Market` витринасы өнімдерді торлы галерея түрінде ұсынады, ал сатып алу алдында растау терезесі қателіктен қорғайды. `Leaderboard` кестесі `Excel` форматына экспортталады — бұл куратор мен әкімшіге студенттердің үлгерімін сыртқы есептерге шығаруға мүмкіндік береді.

13-сурет - Студент дашбордының көрінісі

18

14-суреттер — Q-Market витринасы және үш деңгейлі рейтинг кестесі

Оқу материалдарын меңгеру және тексеру логикасы

Бұл бөлім платформаның функционалдық өзегі болып табылады. Курс

ішіндегі оқу беті бірнеше қойындыдан тұрады: тапсырмалар (classwork),

модульдер (tasks), қатысушылар (people) және хабарламалар лентасы (feed).

Тақырыптар иерархиялық ағаш түрінде ұйымдастырылған, ал әр тақырыптың

күйі түспен белгіленеді: жасыл — аяқталған, көк — ағымдағы, сұр құлып — әлі

ашылмаған. Бұл визуалды иерархия студентке өз прогресін бірден бағалауға

мүмкіндік береді.

Платформаның маңызды инженерлік шешімдерінің бірі — бейне қарауды

бақылау механизмі. Бейнематериал YouTube embed арқылы ойнатылады, бейне

қарау прогресі пайызбен есептеледі және тек 90%-дан астам қаралғанда ғана

теориялық мәтін мен код мысалдары ашылады. Бұл шектеу студентті материалды

жүйелі меңгеруге бағыттап, оқу мазмұнын өткізіп жіберу мәселесін техникалық

деңгейде шешеді.

Тапсырманы тапсыру интерфейсі екі бағаннан тұрады: сол жақта тапсырма

сипаттамасы мен сыныптық пікірлер, оң жақта студенттің жауабы (мәтін, файл

немесе сілтеме) және оқытушыға жеке хабарлама бөлімі. Бағасы мен пікірі сол

интерфейстен тікелей алынады — студент бөгде беттерге өтпейді.

15-сурет - Тақырып бетіндегі бейнеплеер және 90% қарау шегі логикасы

19

16-сурет - Теориялық материалдың ашылуы: мәтін және код мысалдары

17-сурет - Тапсырманы тапсыру интерфейсінің толық көрінісі

Педагогикалық және әкімшілік басқару модульдері

Мұғалім мен куратор панельдері /app/teacher маршруты арқылы жүзеге

асырылған. KPI карточкалары (топтар, студенттер саны, тексерілмеген

жұмыстар) useQueue хуктары арқылы нақты уақытта жаңартылады.

Тапсырмаларды тексеруге арналған «Inbox» жүйесі тексерілмеген және

тексерілген жұмыстарды бөлек қойындыларда ұсынады; жұмыстарды топ

бойынша сүзгілеуге болады. Жаңа тапсырма құру модальды терезесі

(CreateAssignmentModal) Zod валидациясымен қорғалған бірнеше өрістен

тұрады: тақырып, сипаттама, балл, дедлайн, файл.

Әкімші панелі жүйенің жалпы жағдайын мониторингтейтін дашбордтан

және бірнеше басқару модулінен тұрады: пайдаланушылар, курстар, сатып

алулар және қолдау өтінімдері. Аналитика беті уақыт аралығы бойынша

деректерді сүзіп, Excel форматына экспорттауға мүмкіндік береді. Қолдау

тикеттерін басқару модулі (Support Tickets) admin, director және curator рөлдеріне

қолжетімді — әр өтініш жеке тикет ретінде тіркеліп, оның мәртебесі (ашық /

шешілген) бақыланады.

18-суреттер - Мұғалімнің тексерілмеген тапсырмалар инбоксы және бағалау панелі

20

19-сурет - Әкімші дашборды: метрикалар, графиктер және аналитика модулі

Ата-аналық бақылау интерфейсі

Ата-ана панелі көп LMS платформаларында кездеспейтін функционал

болып табылады. Баланың жалпы үлгерімі дөңгелек прогресс диаграммасы

(circular progress chart) арқылы визуалды түрде көрсетіледі. Орташа балл,

меңгерілген тақырыптар саны, рейтингтегі орын және мерзімі өткен

тапсырмалар (қызыл түспен) бір экранда шоғырландырылған — ата-ана

баласының оқу барысын жылдам және толық бағалай алады. Панельдің төменгі бөлігінде әр курс бойынша прогресс жолағы мен орындалмаған тапсырмалардың нақты тізімі беріледі.

20-сурет - Ата-ана панелі

2.3 Серверлік бөлік(Бэкэнд): техникалық іске асыру, деректер базасы және қауіпсіздік

Платформаның серверлік бөлігі Python 3.11+ тілінде FastAPI фреймворкімен жазылған. FastAPI таңдалуының негізгі себептері нақты техникалық артықшылықтарға негізделген: бірінші кезекте асинхронды сұраныстарды өңдеу мүмкіндігі, яғни `async/await` конструкциялары арқылы бір уақытта жүздеген сұранысты бір ағында өңдеуге болады. Бұл дәстүрлі синхронды фреймворктермен салыстырғанда (Django, Flask) өнімділікті арттырады.

Екінші маңызды артықшылық — Pydantic кітапханасымен тығыз интеграция, ол кіріс деректерді Python type hints арқылы автоматты тексереді және қате деректер сервердің ішкі логикасына жетпей тоқтатылады. Uvicorn ASGI сервері FastAPI қосымшасын іске қосып, асинхронды жұмысты жоғары жылдамдықпен қамтамасыз етеді.

Жалпы серверлік бөлік құрылымы `app/` каталогы ішінде модульдер бойынша логикалық бөлінген: `api/routes/` — маршруттар, `core/` — конфигурация мен

21
қауіпсіздік, `models/` — деректер базасы модельдері, `schemas/` — Pydantic схемалары, `services/` — бизнес-логика.

21-сурет – «Qazaq IT Academy» платформасындағы FastAPI сұранысты өңдеу тізбегінің схемасы

Авторизация жүйесінің техникалық іске асырылуы

Пайдаланушы кіру формасын толтырып жібергенде, серверлік бөлікте күрделі тексеру тізбегі іске қосылады. `/api/v1/auth/login` эндпоинтіне POST сұранысы келген кезде `auth.py` модулі бірнеше операцияны дәйекті орындайды.

22-сурет – Платформа JWT негізделген аутентификация, RBAC ағынының схемасы

Алдымен енгізілген email мекенжайы тазаланады: `.strip()` және `.lower()` операциялары артық бос орындар мен бас әріптерді жояды. Бұл пайдаланушы «User@Gmail.COM» деп жазса да, «user@gmail.com» деп жазса да бірдей аккаунтты табуды қамтамасыз етеді. Содан кейін жиі кездесетін домен қателерін автоматты түзету механизмі жұмыс істейді — мысалы «gmial.com» орнына «gmail.com» ұсынылады.

23-сурет – `backend/app/api/routes/auth.py` - файлындағы `/login` маршрутының бағдарламалық коды

22

Email тазаланғаннан кейін SQLAlchemy ORM арқылы `users` кестесінен пайдаланушы іздейді. Егер пайдаланушы табылмаса немесе пароль сәйкес келмесе, жүйе HTTP 401 Unauthorized қатесін қайтарады. Қауіпсіздік тұрғысынан маңызды мәселе: қате хабарламасы «пайдаланушы табылмады» немесе «пароль қате» деп жазылмайды — екі жағдайда да бірдей жалпы хабарлама беріледі. Бұл шабуылдаушының тіркелген email-дерді анықтауына жол бермейді. Сәтті кіру кезінде `log_activity` функциясы арқылы әрекет журналға жазылады, деректер базасына `commit` орындалады, және JWT токени генерацияланып клиентке қайтарылады.

JSON Web Token технологиясы платформада сессияларды серверлік жадта сақтамай-ақ пайдаланушы аутентификациясын жүзеге асыруға мүмкіндік береді.

Бұл stateless архитектура деп аталады және масштабталу үшін маңызды

артықшылық береді.

JWT токени үш бөліктен тұрады: header (алгоритм типі), payload

(пайдаланушы деректері) және signature (жасырын кілтпен қол қойылған).

Payload ішінде пайдаланушының id, email, role және exp (токеннің жарамдылық мерзімі) өрістері сақталады. Токен python-jose кітапханасы арқылы HS256 алгоритмі мен SECRET_KEY арқылы қол қойылады.

Құпия сөздерді қорғаудың криптографиялық тетіктері: bcrypt алгоритмі

Ақпараттық қауіпсіздікте негізгі талаптардың бірі — құпия сөздерді ашық

түрде сақтамау. Себебі деректер базасы бұзылған жағдайда бұл үлкен қауіп

тудыруы мүмкін. Осыған байланысты «Qazaq IT Academy» платформасында

bcrypt алгоритмі қолданылады. Ол құпия сөздерді хэштеп сақтайды және әр мәнге автоматты түрде salt қосады. Бұл rainbow table сияқты шабуылдарды іс жүзінде тиімсіз етеді.

Құпия сөздерді хэштеу және тексеру функциялары орталықтандырылған

түрде security.py модулінде анықталған.

24-сурет – backend/app/core/security.py - файлындағы құпия сөзді хеширлеу және тексеру функциялары

Бұл суретте security.py файлының 9–17 жолдар аралығы көрсетілген. 9-

жолда CryptContext объектісі жасалып, хэштеу схемасы ретінде bcrypt

таңдалады, ал deprecated="auto" ескі алгоритмдерді автоматты түрде өңдеуге

мүмкіндік береді. 12–13 жолдарда get_password_hash функциясы анықталған: ол

құпия сөзді алып, hash() арқылы bcrypt-хэшке айналдырады, әрі әр жолы

кездейсоқ salt қосылады. 16–17 жолдарда verify_password функциясы

орналасқан, ол енгізілген құпия сөзді сақталған хэшпен салыстырып, сәйкестігін тексереді.

23

Деректер базасы интерфейсінде users кестесінің мазмұнын қарасақ,

password_hash бағанында 2b\$12 форматындағы ұзын жолдар бар, бұл bcrypt

алгоритмінің қолданылғанын және оның cost factor мәні 12 екенін білдіреді.

Әрбір хэштің соңғы бөлігі әртүрлі, себебі оған salt қосылады. Бұл бірдей құпия

сөздердің де әртүрлі хэшке айналуын қамтамасыз етеді. Осылайша, құпия сөздер

ашық күйде сақталмайды және деректер базасына тікелей қолжетімділік

болғанның өзінде оларды қалпына келтіру мүмкін емес.

Рөлдерге негізделген қолжетімділік (RBAC) жүйесінің іске асырылуы

Клиенттік бөлікте рөлге байланысты бағыттау логикасы жұмыс істейді —

мысалы, студент /app/courses бетіне кіргенде рөлін тексеріп тиісті бетке

бағыттайды. Бұл клиенттік бөлік деңгейіндегі тексеру ғана. Шын мәнінде нақты

деректерді қорғау сервер жағында орындалады.

permissions.py файлында рөлге арналған тексеру функциялары

орталықтандырылған. Бұл функциялар FastAPI Depends механизмі арқылы

маршрут анықтамасында параметр ретінде жазылады. Сұраныс маршрутқа

жеткен кезде FastAPI алдымен Depends функцияларын дәйекті орындап, шарттар

орындалмаса контроллер функциясын мүлде шақырмайды.

25-сурет – backend/app/api/permissions.py: check_is_teacher, check_is_admin,

check_is_curator функцияларының коды

Мысалы, мұғалімге арналған маршруттарда @router.get("/teacher/groups",

dependencies=[Depends(check_is_teacher)]) жазылады. Осы маршрутқа студент

рөлімен кіруге тырысса, серверлік бөлік 403 Forbidden қайтарады, деректер

мүлде жіберілмейді. Бұл клиенттік бөліктің интерфейсінде кнопкалар

жасырылғанымен API-ға тікелей сұраныс жіберу жағдайында да деректерді

қорғайды.

24

26-сурет – backend/app/core/security.py, 9–17 жолдар: CryptContext конфигурациясы, get_password_hash және verify_password функциялары

Salt механизмінің маңызды мәні мынада: бірдей «password123» паролын екі студент таңдаса да, олардың деректер базасындағы хэш мәндері мүлде басқаша болады. Бұл rainbow table шабуылдарын — яғни алдын ала есептелген хэш кестелерін пайдаланатын шабуылдарды — іс жүзінде тиімсіз етеді [11]. Kіру кезінде verify_password функциясы енгізілген парольді сақталған хэшпен passlib арқылы салыстырады — деректер базасынан хэш алынып, ол кері айналдырылмайды, тек салыстыру орындалады.

27-сурет – Деректер базасы клиентінің көрінісі: users кестесі ашық, password_hash бағанында 2b\$12 ... форматындағы хэш мәндері көрсетілген

Деректер базасы архитектурасы және SQLAlchemy ORM жұмысы

Клиенттік бөліктегі барлық визуалды деректер — курс карточкалары, прогресс жолақтары, рейтинг кестесі, тапсырмалар тізімі — PostgreSQL деректер базасынан алынады. Деректер базасымен жұмыс тікелей SQL сұраныстары арқылы емес, SQLAlchemy ORM (Object-Relational Mapping) арқылы орындалады. Бұл тәсілде Python кластары деректер базасы кестелерінің объектілік моделі ретінде жұмыс істейді.

app/models/ каталогында барлық кесте модельдері орналасқан. User моделі users кестесін сипаттайды: id (Primary Key), email (Unique), password_hash, role, created_at өрістерімен.

25

28-сурет – backend/app/models/user.py файлындағы пайдаланушыларды сипаттайтын User SQLAlchemy моделі.

Course моделі courses кестесін сипаттайды:

29-сурет – backend/app/models/course.py файлындағы курстардың құрылымын анықтайтын Course моделі.

Ал Enrollment моделі enrollments кестесін модельдейді. Кестелер арасындағы байланыстар relationship() арқылы анықталып, foreign key шектеулерімен қамтамасыз етіледі.

30-сурет – backend/app/models/enrollment.py файлындағы кестелер арасындағы байланыстар мен ForeignKey шектеулері.

ORM қолдану SQL-инъекция шабуылдарынан тиімді қорғайды [10].

Дәстүрлі тәсілде SQL сұранысы жолдарды біріктіру арқылы жасалса, шабуылдаушы SQL коды жазып деректер базасын бұза алады. SQLAlchemy параметрленген сұраныстарды қолданады: db.query(User).filter(User.email == email).first() — мұнда email мәні ешқашан SQL кодының бөлігі ретінде орындалмайды.

Деректер базасы сессияларын басқару get_db() генераторы арқылы жүзеге асады. Бұл функция FastAPI маршруттарында Depends арқылы инъекцияланады.

26

try/finally блогы сессияның сұраныс аяқталғаннан кейін міндетті түрде жабылуын қамтамасыз етеді — қате болған жағдайда да сессия «ашық» күйде қалмайды.

PostgreSQL қосылу пулы database.py файлында create_engine арқылы конфигурацияланған. pool_pre_ping=True параметрі бос тұрған қосылыстардың үзілмегенін тексереді, бұл ұзақ уақыт бос тұрған қосылыс пайдаланылғанда пайда болатын TimeoutError қателерін болдырмайды.

Курстар жүйесінің серверлік бөлік логикасы

Студент «Менің курстарым» бетін ашқанда, Next.js

/api/v1/courses/my/enrollments эндпоинтіне GET сұранысы жібереді. Серверлік

бөлікте courses.py маршрут файлы бұл сұранысты қабылдап, алдымен

get_current_user арқылы токенді тексереді, содан кейін деректер базасынан тек

сол пайдаланушыға тіркелген курстарды шығарады.

31-сурет – backend/app/api/routes/courses.py файлындағы студенттің курстар тізімін

joinedload арқылы оңтайландырылған түрде алу эндпоинті

Курс ішіндегі тақырыптардың ашылу шарттары да серверлік бөлік

деңгейінде тексеріледі. Студент тақырыпты ашқан кезде жүйе алдыңғы

тақырыптың аяқталғанын student_progress кестесінен тексереді. Егер алдыңғы

тақырып completed статусында болмаса, сервер 403 немесе арнайы қате

қайтарады.

Бейне қарау прогресін бақылау механизмі

Студент бейнені қараған кезде, белгілі бір уақыт аралықтарда

/api/v1/topics/{id}/video-progress эндпоинтіне PATCH сұраныстары жіберіледі.

Серверлік бөлік алынған пайыз мәнін student_progress кестесіне жазады. Бейне

қарау пайызы 90%-дан асқан кезде теориялық материалдың қолжетімділігі

автоматты түрде белсендіріледі — бұл логика сервер жағында орындалады,

клиенттік бөлік тек серверден статус алып көрсетеді.

32-сурет – backend/app/api/routes/progress.py файлындағы бейне қарау прогресін

жаңарту және теориялық материалды ашу шарты

27

Тапсырманы тапсыру және бағалау жүйесінің серверлік бөлік логикасы

Студент тапсырманы тапсырған кезде /api/v1/assignments/{id}/submit POST

сұранысы жіберіледі. Серверлік бөлік файлды /uploads каталогына сақтап,

submissions кестесіне жазба жасайды: student_id, assignment_id, submitted_at,

file_path, status (submitted → reviewed), grade, feedback өрістерімен. Мұғалім

тапсырманы тексеріп баға қойған кезде /api/v1/submissions/{id}/grade PATCH

сұранысы орындалады.

33-сурет – backend/app/api/routes/assignments.py файлындағы тапсырманы серверге

жүктеу және мұғалімнің баға қою функциялары.

Q-coin транзакция жүйесінің техникалық іске асырылуы

Клиенттік бөліктің дашбордында Q-coin балансы нақты уақытта көрінеді.

Серверлік бөлікте барлық Q-coin операциялары q_coin_transactions кестесінде

жазылады. add_coins функциясы монеталарды есептегенде алдымен transactions

тарихынан сол оқиға үшін монеталардың бұрын берілген-берілмегенін тексереді

— «бір рет есептеу» ережесі осылай іске асырылады. Бұл тексеру маңызды:

пайдаланушы сауалнама жібергенде немесе желі проблемасы кезінде сұраныс

қайталанса да Q-coin қайталап есептелмейді.

34-суреттер - backend/app/services/coins.py файлындағы Q-coin транзакцияларын

атомарлы add_coins және spend_coins, яғни есептеу және қайталап беруді тексеру

логикасы.

spend_coins функциясы сатып алу кезінде алдымен балансты тексереді —

жеткіліксіз болса транзакция орындалмайды. Транзакцияларды атомарлы

орындау үшін PostgreSQL транзакция механизмі қолданылады: монеталарды

шегеру мен тапсырыс жасалуы бір транзакция ішінде орындалады, яғни бір

операция сәтсіз болса екіншісі де орындалмайды.

Rate limiting және DDoS қорғанысы

28

Логин эндпоинтіне rate limiting орнатылған. Бір IP мекенжайынан немесе email-ден белгілі уақыт аралығында жіберілетін сұраныстар саны шектеледі. Бұл brute-force шабуылдарының алдын алады — шабуылдаушы парольдерді автоматты сынап көре алмайды. Лимиттен асқан жағдайда 429 Too Many Requests қатесі қайтарылады.

35-сурет - backend/app/api/routes/auth.py файлындағы логин эндпоинті және brute-force шабуылдарынан қорғайтын rate limiting декораторы.

/api/v1/payments/kaspi/pay эндпоинтіне сәтті төлем расталған кезде

серверлік бөлік бірнеше операцияны дәйекті орындайды: students кестесінде жаңа пайдаланушы жазбасы жасалады, кездейсоқ пароль генерацияланып bcrypt арқылы хэшке айналдырылады, курсқа тіркелу enrollments кестесіне жазылады, және SMTP сервісі арқылы студентке email жіберіледі — онда кіру деректері мен платформаға сілтеме қамтылады.

36-суреттер - backend/app/api/routes/applications.py файлындағы Kaspi төлемін өңдеу және автоматты аккаунт жасау логикасы.

Конфигурация және ортаны басқару

Барлық сезімтал деректер — SECRET_KEY, DATABASE_URL, OpenAI API

Key, Gemini API Key, SMTP параметрлері — backend/app/core/config.py файлы

арқылы орта айнымалыларынан (environment variables) оқылады. Бастапқы кодқа тікелей жазылмайды. Өндірістік ортада бұл мәндер .env файлында немесе Docker secrets арқылы беріледі.

29

37-сурет - backend/app/core/config.py файлындағы жүйелік баптаулар мен құпия деректерді .env файлынан оқу механизмі.

Маңызды мәселе: SECRET_KEY тұрақты мән ретінде .env-те берілмесе, secrets.token_urlsafe(64) арқылы автоматты генерацияланады. Бұл сервер қайта іске қосылған сайын жаңа кілт пайда болып, бұрынғы барлық JWT токендерінің жарамсыз болуына әкеледі. Сондықтан өндірістік ортада тұрақты SECRET_KEY міндетті түрде орнатылуы тиіс.

Файлдарды сақтау жүйесі

Студенттердің жүктеген файлдары — тапсырмалар, материалдар —

сервердің /uploads каталогында Docker volume арқылы сақталады. Бұл

деректердің контейнер жойылса да жоғалмауын қамтамасыз етеді, өйткені

volume хост машинасында физикалық дискіге тікелей байланысқан. Файл жүктеу

кезінде серверлік бөлік файл форматын тексеріп, қауіпті файл типтерін (.exe, .sh

және т.б.) блоктайды. Файл атауы UUID форматындағы кездейсоқ жолға

ауыстырылады — бұл атау соқтығысуларының және бастапқы файл атауы

арқылы ақпаратты ашып алудың алдын алады.

38-сурет - backend/app/api/routes/assignments.py файлындағы файлдарды жүктеу, форматты тексеру және UUID арқылы сақтау логикасы.

Деректер базасының резервтік көшіруі

Өндірістік ортада PostgreSQL деректер базасының автоматты резервтік

көшіруі (backup) күн сайын орындалады. pg_dump утилиті арқылы деректер

базасының толық суреті жасалып, қауіпсіз орында сақталады. Резервтік

деректерді қалпына келтіру pg_restore арқылы орындалады. Бұл механизм жүйе

ақауы жағдайында деректердің жоғалып кетпеуін кепілдендіреді.

30

39-сурет - RBAC схемасы: рөлдер мен олардың қолжетімді маршруттары

40-сурет - backend/app/core/config.py файлында лимиттер slowapi кітапханасы арқылы жасалған коды

41-сурет - docker-compose.yml файлындағы backend сервисінің
environmentайнымалылары мен rate limiting баптауы және Docker volume файл орны

2.4 Жасанды интеллект модульдерінің бағдарламалық-техникалық
реализациясы

«Qazaq IT Academy» платформасындағы жасанды интеллект функциялары

жай қосымша мүмкіндік ретінде емес, оқу процесінің архитектуралық ядросы
ретінде жобаланған. Барлық ЖИ операцияларының логикасы

backend/app/services/ai_service.py модулінде орталықтандырылған, ал жеке
функционалды модульдер /backend/app/api/v1/endpoints/ директориясында бөлек
эндпоинттер түрінде іске асырылған.

Орталықтандырылған ЖИ сервисінің архитектурасы және резервтеу
жүйесі

Жүйеде Service Wrapper паттерні қолданылған. Бұл тәсіл платформаның
бизнес-логикасын нақты бір модельдің API спецификациясынан оқшаулайды —
яғни модельді ауыстырған жағдайда бизнес-логикаға өзгеріс енгізу талап
етілмейді.

Жүйенің жоғары қолжетімділігін қамтамасыз ету үшін Multi-LLM Failover
стратегиясы іске асырылған. Негізгі модель ретінде OpenAI GPT-4o
қолданылады — күрделі аналитика, код талдау және ашық жауаптарды бағалау
үшін. Резервтік модель ретінде Google Gemini 1.5 Flash орнатылған.

get_ai_response() функциясы OpenAI-дан 429 (Rate Limit Exceeded) немесе 500
(Server Error) қатесі қайтарылған жағдайда автоматты түрде fallback_to_gemini()
функциясын шақырады. Пайдаланушы бұл ауысуды байқамайды — жауап
кідіріссіз қайтарылады.

31
42-сурет — backend/app/services/ai_service.py файлындағы get_ai_response және
fallback_to_gemini функцияларының коды. Көрсету керек: try-except блогы және
провайдерлерді ауыстыру логикасы

Context-Aware Prompting: деректерді динамикалық интеграциялау
Платформадағы ЖИ ассистенті қарапайым чат-бот емес — ол студенттің
ағымдағы оқу деректерін ескере отырып жауап беретін контекстік жүйе болып
табылады. Студент сұрау жібергенде, жүйе жауап қайтармас бұрын
get_user_context(user_id) функциясын орындайды. Бұл функция келесі деректерді
деректер базасынан жинайды:

- Enrollments және Grades кестелерінен — студенттің орташа балы және
аяқталған модульдер саны;
- TestHistory кестесінен—соңғы5 тесттежіберілген қателердің тақырыптық
топтамасы (GROUP BY topic);
- ActivityLogs кестесінен — платформадағы белсенділік уақыты мен оқу
қарқыны.

Жиналған деректер JSON форматында System Prompt-қа динамикалық түрде
енгізіледі. Промпт құрылымы келесідей:

Осылайша ЖИ «сен қазір Python функциялары тақырыбын оқып жатырсың
және соңғы тестте 65% алдың» деген деңгейде дербестендірілген кеңес береді.

Диалог тарихы AIChatHistory кестесінде сақталады, бұл сессиялар арасында
контекстің үзілмеуін қамтамасыз етеді.

Студенттердің ЖИ-ды тест сұрақтарының дайын жауабын алу үшін
пайдалану мәселесі екі деңгейде шешілген:

Бірінші деңгей — Regex Filtering. Студент сұранысы алдымен тұрақты
өрнектер (regex) арқылы тексеріледі. «жауап», «нұсқа», «тест», «answer»,

«option» сияқты кілт сөздер анықталса, сұраныс екінші деңгейге жіберіледі.

43-сурет – ИИ ассистент интерфейсі: чат терезесі және контекстік хабарлама

32

Екінші деңгей — LLM Guardrail. `is_suspicious_request(prompt)` функциясы

студенттің сұрауын жеңіл модельге жіберіп, оның академиялық адалдықты бұзып

тұрғанын семантикалық деңгейде бағалайды. Егер сұраныс күдікті деп танылса,

ЖИ дайын жауап бермей, студентті материалды өзі оқуға бағыттайтын hint жауап

қайтарады.

Бұл екі деңгейлі тәсіл жай сөздік сүзгіден (keyword filter) мүлде ерекше:

студент сұрағын перифраз арқылы жасырса да, семантикалық тексеру оны

анықтайды.

44-сурет — `backend/app/services/ai_service.py` файлындағы `is_suspicious_request`

функциясының коды. Көрсету керек: regex тексеру блогы және LLM-ге жіберу

логикасы

Аналитикалық модульдер: Weak Topic Analysis, Personal Learning Plan

Weak Topic Analysis модулі `backend/app/services/analytics_service.py`

файлында орналасқан. Алгоритм студенттің тест жауаптарын GROUP BY topic

операторы арқылы деректер базасы деңгейінде топтастырады, қайта-қайта қате

жасалатын тақырыптарды статистикалық ауытқу ретінде анықтайды және тек

осы ауытқу деректерін ЖИ-ға жібереді. ЖИ статистикалық деректерді адам

түсінетін нақты ұсыныстарға айналдырады.

Personal Learning Plan Generation `generate_personal_plan()` функциясы

арқылы іске асырылған. Промпт-инжиниринг деңгейінде ЖИ-ға Chain-of-

Thought (CoT) техникасы берілген — ЖИ жоспарды бірден ұсынбай, алдымен

студенттің жағдайын талдайды, содан кейін қадам-қадаммен негіздеп жоспар

құрады. Бұл жоспардың логикалық дәйектілігін арттырады.

45-сурет – Жеке оқу жоспары интерфейсі

AI Challenge: интерактивті жарыс модулінің іске асырылуы

AI Challenge модулі `backend/app/api/v1/endpoints/ai_challenge.py` файлында

орналасқан. Студент `/api/v1/challenge/start` эндпоинті арқылы жарысты іске

қосады. `generate_challenge_questions()` функциясы тақырыпқа сәйкес сұрақтарды

33

ЖИ арқылы динамикалық генерациялайды — бұл тест базасының жаттанды

болып кетуін болдырмайды.

ЖИ-дың «нақты қарсылас» сияқты ойнауын симуляциялау үшін клиенттік

бөліктің кумулятивті таймер механизмі қолданылған: `setTimeout` таймерлері

студент жауап бергеннен кейін ЖИ-дың «ойлану уақытын» модельдейді. Жарыс

соңында студент өз нәтижесін ЖИ нәтижесімен салыстырып, қай тақырыпта

артта қалғанын нақты көреді. Үздік студенттерге рейтингке сәйкес 100-ден 1000

Q-coin-ге дейін марапат беріледі.

46-сурет — `ai_challenge.py` файлындағы сұрақ генерациясы және ұпай есептеу

функцияларының коды

47-сурет – Ойын экраны (Студент vs AI)

48-сурет — AI Challenge белсенді ойын процесі: сұрақ, жауап нұсқалары, таймер және

қорытынды нәтижелер экраны

Инфрақұрылымдық қорғаныс және сұраныстар санын шектеу

Server-side API keys: Барлық API кілттері (OpenAI, Gemini)

`backend/app/core/config.py` модулі арқылы `.env` файлынан жүктеледі. Клиенттік

бөлікте ешқандай кілт сақталмайды — бұл «Man-in-the-middle» шабуылдарынан

қорғайды.

Backend Throttling: Бір пайдаланушыға сағатына 50 ЖИ сұранысы лимиті қойылған. Лимитті бақылау FixedWindowRateLimiter механизмі арқылы іске

34

асырылған. Лимит асып кеткенде жүйе 429 статус коды қайтарып, пайдаланушыны хабардар етеді.

Chat History Isolation: AIChatHistory кестесіндегі диалог тарихы тек сол студентке және әкімшілік рөлдерге қолжетімді. Серверлік сұраныстарда user_id бойынша фильтрация орындалады, яғни бір студент басқаның диалогын ешқандай жағдайда көре алмайды.

49-сурет — ЖИ функцияларының жиынтық техникалық схемасы: сұраныстың фронтендтен LLM-ге дейінгі толық жолы, деректерді өңдеу кезеңдері және қауіпсіздік деңгейлері

ЖИ модулінің іске асырылу деңгейі

Жобаның аясында жүзеге асырылған, болашаққа жоспарланған және жоба шеңберінен тыс қалдырылған функциялар арасындағы айырмашылықты нақты белгілеу академиялық дәлдік тұрғысынан маңызды. Төмендегі кестеде ЖИ модулінің функционалдық жағдайы жинақталған.

3-кесте — ЖИ модулінің функционалдық жағдайы

Функция Сипаты Ескерту

Контекстік чат-ассистент Іске асырылған OpenAI GPT-4o + Gemini fallback

Anti-Cheat (Regex + LLM

Guardrail)

Іске асырылған Екі деңгейлі сүзу

Weak Topic Analysis Іске асырылған GROUP BY topic + LLM

түсіндірмесі

Personal Learning Plan Іске асырылған Chain-of-Thought промптинг

AI Challenge жарысы Іске асырылған Динамикалық сұрақ генерациясы

Ашық жауаптарды

автоматты бағалау

Жартылай LLM ұсыныс береді, мұғалім

растайды

Predictive Analytics (үлгерім

болжауы)

Жоспарға енгізілген Болашақ дамыту бағыты

Redis кэш қабаты Іске асырылған Масштабтау кезінде енгізіледі

Дербес ML модельдерін

оқыту

Іске асырылмаған LLM API-сервистері тиімдірек деп

танылды

35

Осылайша, екінші тарауда «Qazaq IT Academy» платформасының үш деңгейлі архитектурасы, Docker орналастыру схемасы, клиенттік бөліктің 40-тан астам беті мен рөлдік интерфейстері, серверлік бөліктің авторизация, RBAC, деректер базасы және қауіпсіздік механизмдері, сондай-ақ ЖИ модулінің негізгі функциялары (Context-Aware Prompting, Anti-Cheat, Weak Topic Analysis, Personal Learning Plan, AI Challenge) толық сипатталды. Бұл техникалық шешімдер үшінші тарауда пайдаланушы тұрғысынан сипатталатын жұмыс сценарийлерінің негізі болып табылады.

36

3 Пайдаланушы нұсқаулығы

3.1 Студентке арналған нұсқаулық: платформамен толық жұмыс

циклі және ИИ мүмкіндіктерін пайдалану

Платформаның тұтас жұмыс циклін студент тұрғысынан түсіну үшін оның алғашқы хабарласудан бастап курсты аяқтауға дейінгі барлық кезеңдерін дәйекті түрде қарастыру маңызды. Төменде студенттің платформамен жұмысының толық сценарийі рөлдік логика негізінде сипатталған.

Курсты сатып алу және аккаунтты алу кезеңі

Студент алдымен платформаның басты бетіне кіріп, курстар каталогімен танысады. Каталогта санаттар бойынша сүзгілеу мүмкіндігі бар, әр курс карточкасында атауы, сипаттамасы, ұзақтығы мен бағасы берілген. Студент қажетті курсты таңдап, Kaspi Pay арқылы төлем жасайды. Төлем сәтті аяқталғаннан кейін жүйе автоматты түрде аккаунт жасайды және кіру деректерін (email мен құпия сөз) студенттің Gmail-іне жібереді. Бұл алғашқы кіру деректерін студент міндетті түрде сақтап қояуы керек.

50-сурет - Курсты сатып алу беті

51-сурет - Kaspi Pay төлем интерфейсі

Жүйеге кіру және алғашқы кез

Студент Gmail-ден алған email мен паролін платформаның кіру бетіне енгізеді. Жүйе деректерді тексеріп, студентті жеке дашбордына бағыттайды.

Алғашқы кезде дашборд негізінен бос болады — курс карточкасы көрінгенімен ішіндегі мазмұнға қолжетімділік жоқ. Бұл қалыпты жағдай: студентті топқа қосу әкімші немесе куратор арқылы орындалады. Топқа қосылғаннан кейін студентке хабарландыру келіп, дашборд толық функционалды режимге ауысады — сабақтар, тапсырмалар, ЖИ модульдері, Q Market және барлық мүмкіндіктер ашылады.

Курспен оқу процесі

37

Білім алушы «Менің курстарым» бөлімінде тіркелген пәндер тізімінен қажетті курсты таңдайды. Курс ішінде модульдер мен тақырыптар иерархиялық тізімде берілген. Тақырыптар бірінен кейін бірі ашылады — алдыңғы тақырыпты аяқтамай келесісіне өту мүмкін емес, бұл материалды жүйелі меңгеруге бағыттайды.

52-сурет – «Мен өтетін курстар» бетінің көрінісі

Нақты тақырыптың оқу беті бірнеше кезеңнен тұрады.

Бірінші кезең — бейнематериал. YouTube embed арқылы ойнатылатын бейнені студент кемінде 90% қарағаннан кейін ғана теориялық бөлім ашылады.

Бұл шектеу студентті материалды жүйелі меңгеруге бағыттайды және оқу мазмұнын өткізіп жіберуді техникалық деңгейде болдырмайды.

Екінші кезең — теория. Мәтіндік материалды оқып болғаннан кейін конспект жазу өрісі белсендіріледі. Студент өз жазбасын сол жерде жазып сақтай алады

Үшінші кезең — тапсырма. Студент практикалық тапсырманы орындап, жауабын мәтін немесе файл түрінде жібереді. Оқытушы жұмысты тексеріп, баға мен пікір қалдырады. Тапсырманың статусы нақты белгіленеді: берілген,

тексерілуде, бағаланған.

53-сурет - Тақырып бетіндегі бейнеплеер және Бейнені қарау шегі (90%)

54-сурет - Теориялық материалдың ашылуы: мәтіндік контент және код мысалдары

55-сурет – Тақырып бойынша конспект және үй тапсырмалары блогы

38

56-сурет – Курс бойынша тапсырма тізбегі

ЖИ-ассистентімен жұмыс

Платформадағы маңызды ЖИ функцияларының бірі — контекстік чат-

ассистент. Студент интерфейстің бұрышында орналасқан ЖИ негізіндегі

виджетті кез келген сәтте іске қосып, өз сұрағын енгізе алады. Ассистенттің

басты ерекшелігі мынада: ол студенттің ағымдағы оқу деректерін біледі.

Мысалы, «Мен осы тақырыпты түсінбей жатырмын» деген сұраққа жалпы жауап

емес, нақты тақырып контекстіндегі түсіндірме береді. «Мен қай тақырыпты

қайталауым керек?» деген сұраққа студенттің соңғы тест нәтижелеріне сүйеніп

нақты тақырып атайды. ЖИ-дың жауаптары бірте-бірте экранға шығарылады

(streaming режимі), бұл табиғи сұхбат атмосферасын тудырады.

57-сурет – ИИ ассистент чат терезесі, контекстік жауап мысалы

Чат тарихы сессиялар арасында сақталады, сондықтан студент ертеңгі күні

платформаға қайта кіргенде бұрынғы сұхбатты жалғастыра алады. Бұл мүмкіндік

студентке ЖИ-ды оқу бойынша тұрақты кеңесші ретінде пайдалануға жол

ашады.

Жеке ЖИ оқу жоспарын жасау

Студент дашборд немесе арнайы бөлімнен «Жеке жоспар жасау»

функциясын іске қосады. ЖИ студенттің барлық оқу деректерін — тест

нәтижелерін, тапсырмалардың орындалу уақытын, белсенділік динамикасын,

осал тақырыптарды — кешенді талдайды. Талдау нәтижесінде студентке

дербестендірілген оқу жоспары ұсынылады. Жоспарда нақтылық маңызды: «осы

аптада функциялар тақырыбына 3 сағат бөл, оны аяқтағаннан кейін объектілерге

кіріс», «сенің оқу қарқыныңа қарай курсты 6 апта ішінде аяқтау мүмкін» деген

мазмұнда практикалық ұсыныстар беріледі.

39

58-сурет – Жеке ИИ жоспарының деңгей таңдау интерфейсі

59-сурет – Жеке ИИ жоспарының нәтиже интерфейсі

Білімдегі олқылықтарды анықтау

Студент дашбордының аналитика бөлімінде ЖИ оның тест тарихын талдап,

қай тақырыптарда жүйелі қате жасалатынын визуалды түрде көрсетеді. Мысалы,

Python тілін оқитын студент үшін «цикл операторлары», «рекурсия», «файлмен

жұмыс» тақырыптары осал ретінде белгіленуі мүмкін. ЖИ осал тақырыптар

бойынша нақты кеңес береді және сол тақырыптарды қайта қарауға тікелей

сілтемелер ұсынады. Бұл мүмкіндік студентке өз білімінің картасын нақты көріп,

мақсатты түрде жаттығуға мүмкіндік береді.

AI Challenge — ЖИ-мен білім жарысы

Платформадағы тартымды ЖИ модульдерінің бірі — жарыс форматы.

Студент «AI Challenge» бөліміне кіріп, курс пен тақырыпты таңдайды. Жарыс

режимі таңдалады: Quiz (уақыт шектеулі сұрақтар) немесе Flashcard (түсінік

карточкалары). «Бастау» батырмасы басылғаннан кейін жарыс іске қосылады.

60-сурет – «AI Challenge» бөлімінің интерфейсі

Quiz режимінде сұрақтар динамикалық генерацияланады — ЖИ тақырып

мазмұнына сүйеніп, әр сессияда жаңа сұрақтар дайындайды, бұл жаттанды

тесттерге жол бермейді. Студент таймер ішінде жауап береді, ал ЖИ-дың

«жауаптары» бірте-бірте экранда пайда болады — нақты қарсылас сияқты.

Жарыс соңында нәтиже салыстырмасы экранда көрсетіледі: студент пен ЖИ-дың

40

дұрыс жауап пайызы, жауап жылдамдығы және тақырып бойынша жеке

статистика. Егер студент жеңіп шықса, Q-coin марапаты есептеледі.

61-сурет – AI Challenge экраны: студент пен ЖИ нәтижелерін салыстыру

Q-coin жинау және Q Market

Платформада барлық оқу белсенділігі Q-coin жүйесімен байланыстырылған.

Студент келесі әрекеттер үшін Q-coin алады:

- күнделікті жүйеге кіру — 5 Q-coin;
- бейне сабақты толық қарау — 25 Q-coin;
- тест тапсыру — нәтижесіне қарай 200-ден 500-ге дейін Q-coin;
- AI Challenge жеңу — 100-ден 1000-ға дейін Q-coin;
- мұғалім тапсырмасынан жоғары баға алу — 50-ден 500-ге дейін Q-coin.

Жиналған Q-coin-дер Q Market дүкенінде жұмсалады. Дүкенде цифрлық және материалдық сыйлықтар бар. Студент қажетті тауарды таңдап, «Жылдам сатып алу» батырмасын басады, растау терезесінде мәліметтерді тексеріп, сатып алуды бекітеді. Операция нәтижесі «Менің тапсырыстарымда» сақталады.

62-сурет – Q Market дүкенінің витринасы

Рейтинг жүйесі

Leaderboard бетінде барлық студенттер үш топқа бөлінген: үздіктер (Top), орташа деңгей (Middle) және бастаушылар (Low). Студент өзінің рейтингтегі орнын, жинаған ұпайын және басқа студенттермен салыстырманы бірден көреді. Бәсекелестік элементі оқу мотивациясын арттырудың тиімді тетігі ретінде жұмыс істейді.

41

63-сурет – Студенттер Рейтингінің интерфейсі

Сервистік қолдау жүйесі

Студентте кез келген сұрақ немесе техникалық мәселе туындаса, «Қолдау» бөліміндегі тикет жүйесі арқылы өтінім жіберіледі. Форма толтырылып, тиісті курс таңдалып, мәселе сипатталады. Өтінімнің статусы нақты белгіленеді, ал куратор немесе әкімші жауап берген кезде студент хабарландыру алады.

64-сурет – Қолдау қызметі беті

3.2 Басқа пайдаланушыларға арналған нұсқаулық: мұғалім, куратор, әкімші, директор және ата-ана

Бұл бөлімде платформаның қалған бес рөлі үшін жұмыс сценарийлері бірыңғай схема бойынша сипатталады: рөлдің негізгі міндеті → қолжетімді бөлімдер → типтік жұмыс сценарийі → рөлдің жүйедегі нәтижесі.

Мұғалімнің жұмыс циклі

Рөлдің негізгі міндеті: оқу мазмұнын жасау, студенттердің тапсырмаларын тексеру, кері байланыс беру.

Қолжетімді бөлімдер: педагогикалық дашборд, «Мен оқытатын курстар», тапсырмалар инбоксы, аналитика.

Типтік жұмыс сценарийі:

Мұғалім жүйеге кіргеннен кейін педагогикалық дашборд ашылады.

Дашборд мұғалімнің барлық маңызды ақпаратын бір экранда шоғырландырады: тексерілмеген жұмыстар санаушы, жақын дедлайндар тізімі және студенттердің соңғы тапсырмалары. Жұмыс күні әдетте тексерілмеген жұмыстарды қарап шығудан басталады.

42

65-сурет – Мұғалім дашбордының толық көрінісі

«Мен оқытатын курстар» бөлімінде мұғалімнің барлық топтары карточка түрінде берілген. Топ ішіне кіргеннен кейін төрт бөлім ашылады. Тапсырмалар (classwork) бөлімінде оқу мазмұны иерархиялық тізімде ұйымдастырылған.

66-сурет – «Мен оқытатын курстар» беті

Жаңа оқу мазмұнын құру. «Тапсырмалар» бөлімінде мұғалім жаңа оқу

материалдарын жасай алады. Ол үшін осы қойындыға кіріп, «+ Құру» батырмасын таңдайды. Содан кейін қажетті элемент түрі таңдалады: тапсырма, тесті бар тапсырма, материал, сұрақ, жаңа тақырып немесе бұрынғысын қайта пайдалану. Ашылған формада атауы, сипаттамасы, ұпайы, мерзімі және қажет қосымшалар енгізіледі. Соңында «Сақтау» немесе «Жариялау» батырмасы басылады.

67-сурет – «+ Құру» ашылмалы мәзірі

Тапсырмаларды тексеру үшін инбокс жүйесі қолданылады. Мұғалім

«Тексерілмеген» қойындысына кіріп, қажет болса топ бойынша сүзгі қояды.

Таңдалған тапсырманы ашқанда сол жақта студенттер тізімі, оң жақта таңдалған

студенттің жұмысы мен бағалау панелі көрінеді. Мұғалім жұмысты оқып, баға

қойып, пікір жазады. ЖИ мұғалімге ашық жауаптарды алдын ала тексеріп,

бастапқы бағалау ұсынысын бере алады — бұл мұғалімнің жұмыс уақытын

айтарлықтай үнемдейді. Сақталған баға студентке бірден хабарландыру ретінде жіберіледі.

68-сурет – Тексерілмеген тапсырмалар инбоксы

43

Рөлдің жүйедегі нәтижесі оқу материалдарының тұрақты түрде жаңартылуы, студенттік жұмыстардың уақтылы тексерілуі және кері байланыстың берілуі болып табылады. ЖИ-дың бастапқы бағалауы арқылы мұғалімнің жүктемесі азаяды.

Куратордың жұмыс циклі

Рөлдің негізгі міндеті: топтарды басқару, студенттерді топқа қосу және олардың жалпы үлгерімін қадағалау.

Қолжетімді бөлімдер: мұғалім панелімен бірдей бөлімдер + топ басқару модулі + кіру өтінімдері қойындысы.

Типтік жұмыс сценарийі:

Куратор мұғаліммен бірдей педагогикалық функцияларды орындайды, бірақ қосымша топтарды басқару, студенттерді топқа қосу/шығару және топқа қосылу өтінімдерін өңдеу мүмкіндіктеріне ие. Куратор панелінде «Кіру сұраулар» қойындысы арқылы жаңа студенттердің топқа қосылу өтінімдері расталады немесе қабылданбайды.

69-суреттер – Куратор панелінің толық интерфейсі: «Топтар», «Кіру сұраулар»

Рөлдің жүйедегі нәтижесі: студенттер дұрыс топтарға тіркеледі, топ құрамы актуалды күйде сақталады. Куратор платформадағы оқу ұйымдастырушысы ретінде жұмыс істейді.

Әкімші мен директордың жұмыс циклі

Рөлдің негізгі міндеті: жүйені жалпы басқару, пайдаланушыларды тіркеу мен реттеу, аналитиканы бақылау.

Қолжетімді бөлімдер: жүйелік басқару панелі — пайдаланушылар, курстар, аналитика, сатып алулар, қолдау өтінімдері.

Типтік жұмыс сценарийі:

Әкімші және директор жүйелік басқару панелін пайдаланады. Панельдің негізгі бөлімдері: пайдаланушыларды басқару, курстарды басқару, аналитика, сатып алуларды бақылау және қолдау өтінімдерін өңдеу.

Пайдаланушыларды басқару бетінде іздеу және рөл бойынша сүзгілеу мүмкіндігі бар, пайдаланушы мәліметтерін өзгертуге және ата-ана мен бала байланысын реттеуге болады.

Аналитика бетінде уақыт аралығы бойынша платформа статистикасы графиктер түрінде берілген және деректер Excel форматына экспортталады. Бұл

мүмкіндік платформаның жалпы жұмысын сандық тұрғыдан бағалауға

мүмкіндік береді.

Қолдау тикеттерін басқару модулі (Support Tickets) admin, director және curator рөлдеріне қолжетімді — әр өтініш жеке тикет ретінде тіркеліп, оның мәртебесі (ашық / шешілген) бақыланады.

44

70-сурет – Әкімші/Директор басқару панелінің толық көрінісі

Рөлдің жүйедегі нәтижесі: платформаның барлық пайдаланушылары мен процестері орталықтандырылған басқару панелінен реттеледі. Аналитика деректері шешім қабылдауға негіз болады.

Ата-ананың жұмыс циклі

Рөлдің негізгі міндеті: баланың оқу прогресін қашықтан бақылау, үлгерімі туралы толық ақпаратқа қолжетімділік.

Қолжетімді бөлімдер: ата-ана дашборды, балалар тізімі, жеке бала профилі, үлгерім есебі.

Типтік жұмыс сценарийі:

Ата-ана панелі баланың оқу барысын қашықтан бақылауға арналған. Жүйеге кіргеннен кейін ата-ана жиынтық бетіне өтеді. Онда балалар саны, жалпы курстар, алынған сертификаттар және орташа үлгерім пайызы шоғырланған.

Нақты баланы таңдағанда оның толық статистикасы ашылады: дөңгелек прогресс диаграммасы, орташа балл, меңгерілген тақырыптар саны, рейтингтегі орын. Төменде курстар бойынша егжей-тегжейлі үлгерім мен орындалмаған тапсырмалардың нақты тізімі беріледі.

Мерзімі өткен тапсырмалар қызыл түспен, мерзімі жақындаған тапсырмалар сары түспен белгіленеді — бұл ата-анаға баласының жағдайын бірден бағалауға мүмкіндік береді. Қажет болса, үлгерім есебін PDF форматында жүктеп немесе басып шығаруға болады.

71-сурет – Ата-ана панелі

Рөлдің жүйедегі нәтижесі: ата-ана баласының оқу барысын нақты уақытта бақылай алады, мерзімі өткен тапсырмалар туралы хабардар болады және үлгерім есебін кез келген уақытта ала алады.

Осылайша, үшінші тарауда платформаның алты рөлі (студент, мұғалім, куратор, әкімші, директор, ата-ана) үшін жұмыс сценарийлері дәйекті түрде сипатталды. Студенттің сатып алудан ЖИ мүмкіндіктерін пайдалануға дейінгі толық циклі, мұғалім мен куратордың педагогикалық басқаруы, әкімші мен директордың аналитикасы және ата-ананың бақылау функциясы қамтылды. Бұл сценарийлер бірінші тараудағы рөлдік талаптар мен екінші тараудағы техникалық шешімдерге толық сәйкес келеді.

45

Қорытынды

Дипломдық жұмысты орындау барысында қойылған негізгі мақсат — жасанды интеллект мүмкіндіктері интеграцияланған «Qazaq IT Academy» білім беру платформасын жобалау және іске асыру — толықтай жүзеге асырылды.

Мақсатқа жету үшін келесі міндеттер шешілді: клиент-серверлік

архитектура негізінде жүйе жобаланды, Next.js және FastAPI фреймворктері таңдалып, Docker контейнерлері арқылы тұрақты жұмыс қамтамасыз етілді;

PostgreSQL деректер базасы құрылып, пайдаланушылар, курстар, үлгерім және

транзакциялар арасындағы күрделі байланыстар іске асырылды; студент, мұғалім, куратор, әкімші және ата-ана рөлдеріне арналған 40-тан астам дербес бет жасалды; жүйенің қауіпсіздігі JWT аутентификациясы, bcrypt хэштеу, RBAC

рөлдік модель және rate limiting механизмдері арқылы қамтамасыз етілді.

Әзірленген платформаның басты ерекшелігі — жасанды интеллект модулі.

Бұл модуль сыртқы API-ларды жай қосу емес, оқу процесіне терең

интеграцияланған кешенді инженерлік шешім болып табылады. Барлық ЖИ

операцияларының логикасы ai_service.ru модулінде орталықтандырылған, ал

жүйенің тұрақтылығы OpenAI GPT-4o негізгі модель және Google Gemini 1.5

Flash резервтік модель түрінде іске асырылған Multi-LLM Failover

стратегиясымен қамтамасыз етілген.

Платформадағы ЖИ ассистенті студенттің ағымдағы тақырыбын, тест

нәтижелерін және үлгерімін біле отырып, контекстке сай жеке кеңес беретін

ментор ретінде жұмыс істейді. Студенттің әлсіз тұстарын статистикалық талдау

(Weak Topic Analysis) арқылы анықтап, оған дербес оқу траекториясын

генерациялайтын Personal Learning Plan модулі — дәстүрлі оқытудағы жеке

мұғаліммен кеңесудің цифрлық баламасы болып табылады. AI Challenge модулі

геймификация элементі ретінде студент пен ЖИ арасындағы білім жарысын

ұйымдастырып, оқу мотивациясын арттырудың тиімді тетігіне айналды.

Сонымен қатар, студенттердің ЖИ-ды тестжауаптарын алу үшін пайдалануының

алдын алатын екі деңгейлі Anti-Cheat жүйесі — Regex Filtering және LLM

Guardrail — академиялық адалдықты техникалық деңгейде қорғайды.

Болашақта жобаны дамыту бағыттары ретінде жүйені Kubernetes

кластерінде масштабтауды, Redis кэш деңгейін қосуды және студенттердің оқу

нәтижелерін алдын ала болжайтын Predictive Analytics модулін енгізуді

қарастыруға болады.

Қорыта айтқанда, «Qazaq IT Academy» — жасанды интеллект көмегімен әр

студентке жеке көзқарас қалыптастыратын, геймификация арқылы оқу

мотивациясын арттыратын және ата-аналық бақылау мүмкіндігін қамтитын

заманауи білім беру экожүйесі болып табылады.

46

Пайдаланылған әдебиеттер тізімі

1. Al-Fraihat D., Joy M., Sinclair J. Evaluating E-learning systems success: An empirical study // Computers in Human Behavior. – 2020. – Vol. 102. – P. 67–86.

2. Brusilovsky P., Millán E. User Models for Adaptive Hypermedia and Adaptive Educational Systems // The Adaptive Web. – Berlin: Springer, 2007. – P. 3–53.

3. Holmes W., Bialik M., Fadel C. Artificial Intelligence in Education: Promises and Implications for Teaching and Learning. – Boston: Center for Curriculum Redesign, 2019. – 52 p.

4. Luckin R., Holmes W., Griffiths M., Forcier L.B. Intelligence Unleashed: An Argument for AI in Education. – London: Pearson Education, 2016. – 47 p.

5. OpenAI API Documentation [Электрондық ресурс]. – Қолжетімділік режимі: <https://platform.openai.com/docs>

(қаралған күні: 23.01.2026).

6. Google AI for Developers: Gemini API Documentation [Электрондық ресурс]. – Қолжетімділік режимі: <https://ai.google.dev/>

(қаралған күні: 02.03.2026).

7. Moodle.org [Электрондық ресурс]. – Қолжетімділік режимі: <https://moodle.org/> (қаралған күні: 11.01.2026).

8. Coursera [Электрондық ресурс]. – Қолжетімділік режимі: <https://www.coursera.org/> (қаралған күні: 11.01.2026).

9. edX [Электрондық ресурс]. – Қолжетімділік режимі:

<https://www.edx.org/> (қаралған күні: 15.02.2026).

10.Khan Academy [Электрондық ресурс]. – Қолжетімділік

режимі: <https://www.khanacademy.org/>

(қаралған күні: 18.02.2026).

11.FastAPI Documentation [Электрондық ресурс]. – Қолжетімділік

режимі: <https://fastapi.tiangolo.com/> (қаралған

күні: 23.02.2026).

12.PostgreSQL Documentation [Электрондық ресурс]. – Қолжетімділік

режимі: <https://www.postgresql.org/docs/>

(қаралған күні: 23.02.2026).

13.SQLAlchemy Documentation [Электрондық ресурс]. – Қолжетімділік

режимі: <https://docs.sqlalchemy.org/> (қаралған

күні: 23.02.2026).

14.React Documentation [Электрондық ресурс]. – Қолжетімділік

режимі: <https://react.dev/> (қаралған күні: 23.02.2026).

15.Next.js Documentation [Электрондық ресурс]. – Қолжетімділік

режимі: <https://nextjs.org/docs> (қаралған күні:

12.01.2026).

16.TypeScript Documentation [Электрондық ресурс]. – Қолжетімділік

режимі:

<https://www.typescriptlang.org/docs/> (қаралған күні: 02.03.2026).

47

17.Docker Documentation [Электрондық ресурс]. – Қолжетімділік

режимі: <https://docs.docker.com/> (қаралған күні:

02.03.2026).

18.Nginx Documentation [Электрондық ресурс]. – Қолжетімділік

режимі: <https://nginx.org/en/docs/> (қаралған күні:

02.03.2026).

19.OWASP Top 10: Web Application Security Risks [Электрондық ресурс]. –

Қолжетімділік режимі:

<https://owasp.org/www-project-top-ten/> (қаралған күні: 02.02.2026).